
S/W Detailed Level Design

Project Name	Currency Exchange System — Release 2		
Block Name	Compile Commander		
Author	Magnus Jaaska	Approver	
Team	Compile Commander		

S/W Detailed Level Design

This document represents Detailed Level Design (DLD). It describes the detailed system design and implementation plan in alignment with Agile principles. The DLD is updated incrementally with each release to reflect system evolution.

Contents

1. Overview.....	Error! Bookmark not defined.
2. System Overview / Architectural Context.....	Error! Bookmark not defined.
3. UML Class Diagram (Technical Design)	Error! Bookmark not defined.
4. Class Specifications	Error! Bookmark not defined.
5. Interfaces and Abstractions	Error! Bookmark not defined.
6. Function Responsibilities	Error! Bookmark not defined.
7. Operation Flow	Error! Bookmark not defined.
8. Enumerations & Constants	Error! Bookmark not defined.
9. Validation Rules & Future Work	Error! Bookmark not defined.
10. Traceability Matrix.....	Error! Bookmark not defined.
11. Code Structure and File Mapping	Error! Bookmark not defined.
12. Revision History	Error! Bookmark not defined.
13. Validation Rules & Preconditions/Postconditions	13
14. Behavioral Models	14
15. Error & Exception Handling Policy	15
16. Revision History	16

S/W Detailed Level Design

■ Revision History

Version	Date	Revised contents	Author	Approver
1.0	28.10.2025	Initial Release 2 DLD; three-layer refactor; interfaces; method responsibilities; traceability.	Magnus Jaaska	

■ Terms and Abbreviations

Term	Description
UI	User Interface (ConsoleUI)
DLD	Detailed Level Design document
SRS	Software Requirements Specification
DIP	Dependency Inversion Principle
SRP	Single Responsibility Principle

■ References

1. SW Requirements Specification (docs/release-1/SRS.pdf)
2. Lecture 5–7 (Design for Quality, Modularity, Abstraction, Polymorphism)
3. Example DLD Currency Exchange (reference structure)

1. Overview

The Currency Exchange System automates currency conversion, verifies reserves before payout, records transactions in memory, and provides a daily summary. This DLD defines the Release 2 design: modular three-layer architecture (UI → Logic → Data), class responsibilities, interfaces, function contracts, constants, operation flow, and SRS traceability. Validation, exception handling, and file-based persistence are planned for Release 3.

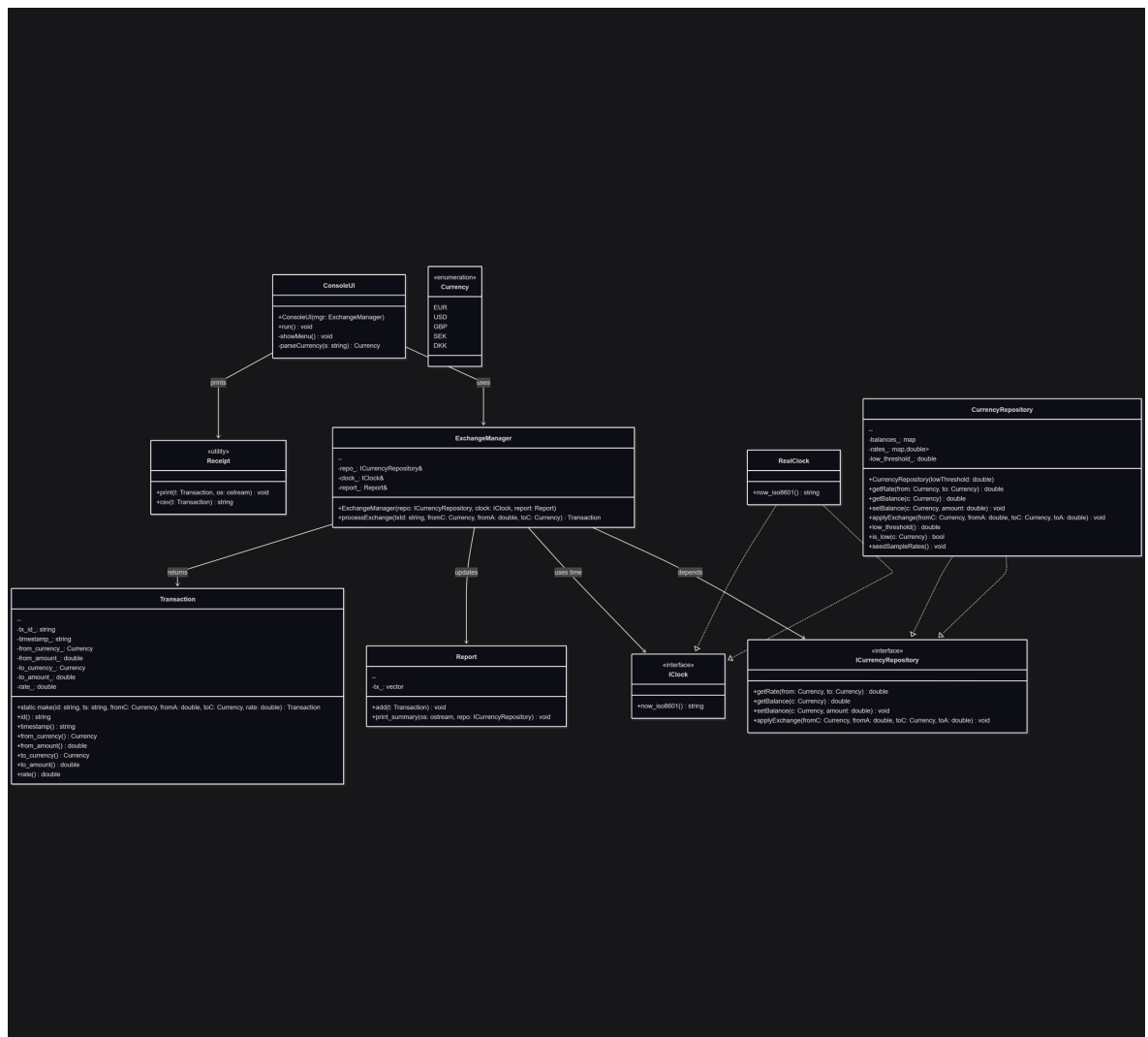
S/W Detailed Level Design

2. System Overview / Architectural Context

- **UI:** ConsoleUI — console interaction & formatting only.
- **Logic:** ExchangeManager — coordinates conversion, reserve checks, and reporting.
- **Data:** ICurrencyRepository / CurrencyRepository — in-memory balances & exchange rates.

Principles: Separation of Concerns, SRP, DIP, encapsulation via header/source split.

3. UML Class Diagram (Technical Design)



4. Class Specifications

Class	Type	Description	Attributes	Methods
ConsoleUI	UI	Console I/O and formatting; no business logic.		run(), parseCurrency(s), showMenu()
ExchangeManager	Logic	Coordinates exchange; checks reserve; updates report.	repo_: ICurrencyRepository& clock_: IClock& report_: Report&	processExchange(txId, fromC, fromA, toC): Transaction
ICurrencyRepository	Interface	Abstracts data access for rates/balances.		getRate(from,to), getBalance(c), setBalance(c,amt), applyExchange(fromC,fromA,toC,toA)
CurrencyRepository	Data	In-memory balances & rates; implements interface.	balances_, rates_, low_threshold_	getRate(), getBalance(), setBalance(), applyExchange(), is_low(c), seedSampleRates()
Transaction	Domain	Immutable exchange record w/ computed to-amount.	tx_id_, timestamp_, from_currency_, from_amount_, to_currency_, to_amount_, rate_	static make(id,ts,fromC,fromA,toC,rate): Transaction
Report	Domain	Aggregates transactions; prints daily summary.	tx_: vector<Transaction>	add(t), print_summary(os, repo)
Receipt	Utility	Formats receipt and CSV line.		print(t, os), csv(t): string
IClock	Interface	Time abstraction.		now_iso8601(): string

S/W Detailed Level Design

RealClock	Service	System clock implementation.	now_iso8601(): string
-----------	---------	------------------------------	-----------------------

5. Interfaces and Abstractions

Interface	Purpose	Key Methods	Planned For (Release)
ICurrencyRepository	Decouple logic from storage; enable File/DB later	getRate(), getBalance(), setBalance(), applyExchange()	R2 (implemented)
IClock	Decouple time for tests	now_iso8601()	R2 (implemented)

6. Function Responsibilities

Class	Method	Purpose	Input	Output	Notes
ExchangeManager	processExchange()	Computes payout, checks reserve, applies exchange, records transaction	txId, fromC, fromA, toC	Transaction	Throws on insufficient reserve/bad rate
CurrencyRepository	applyExchange()	Update balances for from/to	fromC, fromA, toC, toA		Encapsulated state change

Report	add()	Append to daily collection	Transaction		Called after success
Report	print_summary() ()	Emit daily totals/closing balances	ostream, ICurrencyRepository		Presentation kept minimal
Receipt	print()	Human-readable receipt	Transaction, ostream		UI use
Receipt	csv()	CSV transaction line	Transaction	string	For logging later
RealClock	now_iso8601()	Timestamp		string	Local ISO-8601
ConsoleUI	run()	Interactive loop; parse/execute	stdin	stdout	Uses manager; handles errors

7. Operation Flow

1. ConsoleUI reads input ("EUR 100 USD").
2. Parses → ExchangeManager::processExchange().
3. Manager gets rate from ICurrencyRepository, computes payout.

S/W Detailed Level Design

4. Validates reserves → applies exchange in repo.
5. Creates Transaction, adds to Report.
6. UI prints Receipt; session end → Report::print_summary().

8. Enumerations & Constants

Name	Value / Type	Description
enum class Currency	{EUR, USD, GBP, SEK, DKK}	Supported currencies
LOW_THRESHOLD	double = 100.0	Alert threshold for low balances
INITIAL_BALANCES	per-currency doubles	Startup config for interactive run

9. Validation Rules & Future Work

Rule / Planned Feature	Description	Target
Input validation & errors	amount > 0, valid currency codes; clear error messages	R3
Strict reserve enforcement	Block payout > reserve; error codes	R3
Exception policy	try/catch at UI; noexcept in Data	R3
File I/O persistence	Save/load rates, tx, reports	R3
Unit tests	Mock IClock & repo; per-layer tests	R3

10. Traceability Matrix

Requirement (SRS)	Class / Method (DLD)	Notes
RQ-01 Convert currency	ExchangeManager::processExchange()	Core conversion via repo rate
RQ-02 Show receipt	Receipt::print(), ConsoleUI::run()	UI formatting only
RQ-03 Maintain reserves	CurrencyRepository::applyExchange()	Encapsulated data update
RQ-04 Daily report	Report::add(), Report::print_summary()	End-of-day summary
RQ-05 Time stamps	RealClock::now_iso8601()	Abstracted time

11. Code Structure and File Mapping

Class / Module	Files
ConsoleUI	include/console_ui.h, src/console_ui.cpp
ExchangeManager	include/exchange_manager.h, src/exchange_manager.cpp
ICurrencyRepository	include/currency_repository.h (interface)
CurrencyRepository	include/currency_repository.h, src/currency_repository.cpp
Transaction	include/transaction.h, src/transaction.cpp
Report	include/report.h, src/report.cpp
Receipt	include/receipt.h, src/receipt.cpp
IClock / RealClock	include/iclock.h, src/iclock.cpp
Entry point	src/main.cpp
Build	Makefile

Name

Date

S/W Detailed Level Design

12. Revision History

Date	Version	Change Summary	Author
28.10.2025	1.0	Initial DLD created from Release 2 refactor	Magnus Jaaska

Release 3

13. Validation Rules & Preconditions/Postconditions

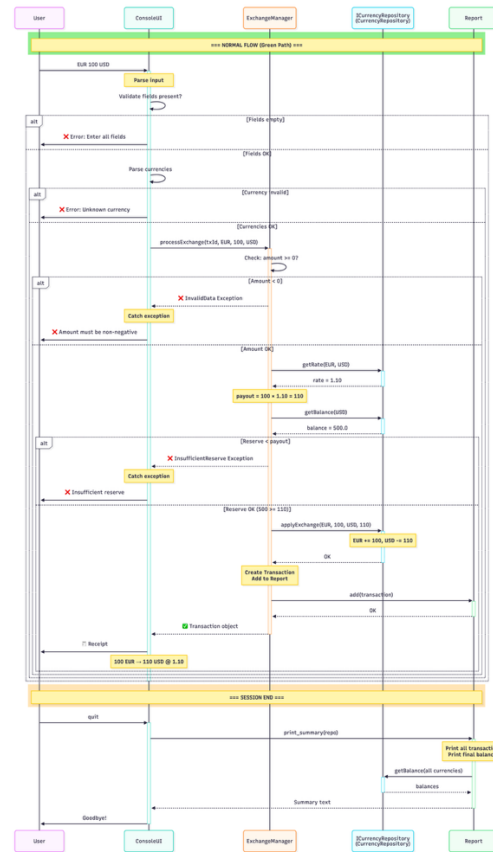
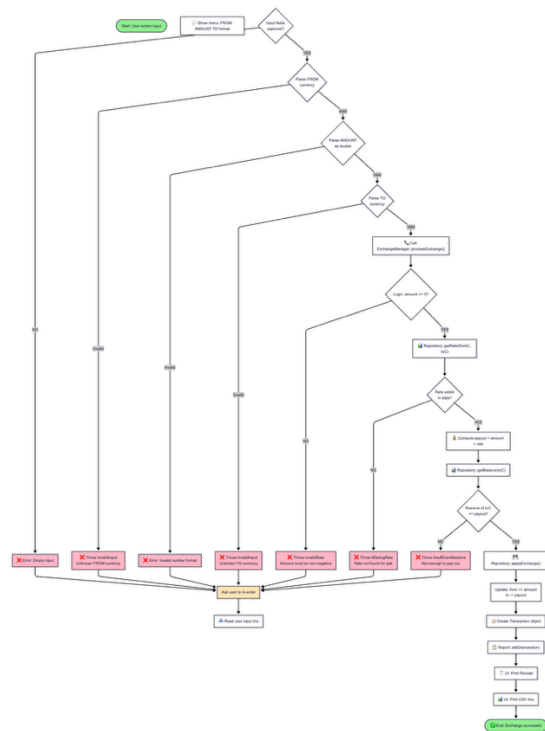
< Describe how your system checks input data and enforces correctness before and after operations. Each method should list clear preconditions (what must be true before it runs) and postconditions (what must be true after it completes). Indicate the layer where validation happens - UI, Logic, or Repository. Use the same class and method names as in Release 2.>

Class	Method	Preconditions	Postconditions	Validation Level (UI/Logic/Repository)	Explanation
ConsoleUI	run()	Input fields present	Input values captured	UI	Validates all required fields are entered
ExchangeManager	processExchange()	fromA >= 0, currency codes valid	Transaction created, balances updated	Logic	Validates business rules before Repository call
CurrencyRepository	getRate	Rate pair exists	Returns rate or throws exception	Repository	Confirms data availability
CurrencyRepository	applyExchange	All validations passed	Balances updated successfully	Repository	Applies validated exchange

S/W Detailed Level Design

14. Behavioral Models

< Attach two UML diagrams that describe one key operation in your system, for example, processing an order, confirming a booking, or updating a record. Both diagrams must describe the same operation to show consistency.



15. Error & Exception Handling Policy

< List all exceptions that may occur in your system. Describe where each one is thrown, where it is caught, what message it shows, and what the program does afterwards.

For this release, use only the three architectural layers: UI, Logic, Repository. >

Exception Type	Thrown By (Layer / Class)	Caught At (Layer)	Message / what()	Default Action (message / stop / retry)
InvalidInput	UI — ConsoleUI input parsing or parseCurrency()	UI	"Please enter all required fields."	Show message → ask user to re-enter
InvalidInput	UI — ConsoleUI parseCurrency()	UI	"Unknown currency: {code}. Supported: EUR, USD, GBP, SEK, DKK"	Show message → ask user to re-enter
InvalidData	Logic — ExchangeManager::processExchange() guard clause	UI	"Amount must be non-negative."	Show message → stop operation
InsufficientReserve	Logic — ExchangeManager::processExchange() reserve check	UI	"Insufficient reserve of {currency} to pay out {amount}."	Show message → stop operation
MissingRate	Repository — CurrencyRepository::getRate()	UI	"Exchange rate not found for pair {from}/{to}."	Show message → stop operation

S/W Detailed Level Design

16. Revision History

<Track document changes across releases. >

Date	Version	Change Summary	Author
28.10.2025	1.0	Initial Release 2 DLD; three-layer refactor; interfaces; method responsibilities; traceability.	Magnus Jaaska
23.11.2025	2.0	Release 3 update: Added Section 13 (Validation Rules), Section 14 (Activity & Sequence Diagrams), Section 15 (Exception Policy). Documented processExchange() as core operation with complete validation, behavior, and error handling flow.	Magnus Jaaska